



دانشگاه شریف

بسته‌ی پروژه‌ی پایان‌ترم دانشجو
درس ساختار کامپیوتر و میکروپروسسور

شرح کامل پروژه، مجموعه‌دستور، برنامه‌ی زمانی و شیوه‌ی نمره‌دهی

طراحی و پیاده‌سازی پردازنده‌ی PARS-16 با نرم‌افزار Digital

ددلاین : ۱۰ مرداد ۱۴۰۵

۲۴ تیر ۱۴۰۵

فهرست مطالب

۱	مقدمه، هدف و قوانین کلی پروژه	۱
۱-۱	این پروژه چیست	۱
۲-۱	اهداف یادگیری	۱
۳-۱	پیش نیازها	۲
۴-۱	نصب و راه اندازی ابزار	۲
۵-۱	صداقت علمی و قوانین تحویل	۳
۱-۵-۱	قالب تحویل	۳
۲	مرجع کامل مجموعه دستور PARS-16	۴
۱-۲	مشخصات کلی (الزامی و ثابت برای همه ی گروه ها)	۴
۲-۲	رجیسترها	۵
۳-۲	مدل حافظه	۵
۴-۲	رجیستر فلگ و اجرای شرطی سراسری	۶
۱-۴-۲	رجیستر فلگ	۶
۲-۴-۲	بیت S: فعال سازی به روز رسانی فلگ	۷
۳-۴-۲	کد شرط دوبیتی (CC)	۷
۴-۴-۲	مقایسه چگونه انجام می شود	۸

۸	۵-۴-۲ الگوی پرش شرطی: چرا دو سیکل کلاک
۸	۶-۴-۲ شبیه‌سازی شرط‌های معکوس (GE، NE، ...)
۹	۵-۲ فرمت‌های دستور
۱۲	۶-۲ جدول کامل دستورات
۱۳	۷-۲ شبه‌دستورها
۱۳	۸-۲ نحو زبان اسمبلی
۱۴	۱-۸-۲ مثال کامل: جمع اعداد 1 تا N
۱۵	۳ راهنمای معماری میکرو و قرارداد رابط اجباری
۱۵	۱-۳ مسیر داده‌ی تک‌سیکل
۱۵	۱-۱-۳ نمای کلی بلوکی
۱۶	۲-۱-۳ ترتیب پیشنهادی ساخت اجزا (مطابق با فصل ??)
۱۷	۲-۳ جدول سیگنال‌های کنترلی
۱۸	۱-۲-۳ پیشنهاد برای ساخت خودِ واحد کنترل
۱۸	۳-۳ قرارداد پین الزام سخت برای نمره‌دهی خودکار
۱۹	۱-۳-۳ نکته‌ی پیاده‌سازی: پورت خواندنِ سوم رجیستر فایل
۲۰	۴-۳ سؤالات متداول طراحی
۲۱	مراجع

فصل ۱

مقدمه، هدف و قوانین کلی پروژه

۱-۱ این پروژه چیست

در این پروژه، در بازه‌ی زمانی سه هفته، یک پردازنده‌ی کامل و کارکننده به نام PARS-16 را طراحی و شبیه‌سازی خواهید کرد. PARS-16 یک مجموعه‌دستور اختصاصی این درس است: نه RISC-V واقعی است و نه MIPS واقعی، اما دقیقاً با همان روحیه‌ی RISC طراحی شده دستورهایی با طول ثابت، فرمت‌های منظم و تعداد کم دستور و یک ویژگی خاص و آموزنده دارد که در بخش ۲-۴ توضیح داده می‌شود: اجرای شرطی سراسری^۱ روی تمام دستورات، نه فقط پرش‌ها.

۲-۱ اهداف یادگیری

- درک این‌که یک مجموعه‌دستور^۲ چگونه به مسیر داده و سیگنال‌های کنترلی نگاشت می‌شود.
- تجربه‌ی عملی ساخت رجیستر فایل، ALU، واحد کنترل و حافظه از قطعات پایه.
- فهم عمیق‌تر مفهوم تک‌سیکل از طریق ساختن (نه فقط خواندن).

^۱ Predicated Execution

^۲ ISA

- آشنایی با یک روش واقعی متفاوت برای شرط‌گذاری در دستورها (اجرای پیش‌بینی‌شده، الگوی معماری ARM) و مقایسه‌ی آن با الگوی رایج‌تر «پرش شرطی» در RISC-V/MIPS.
- تجربه‌ی نوشتن برنامه‌ی اسمبلی برای سخت‌افزاری که خودتان ساخته‌اید.

۱-۳ پیش نیازها

فرض می‌شود مباحث زیر را از پیش می‌دانید: جبر بول و دروازه‌های منطقی، مالتی‌پلکسر و دیکودر، فلیپ‌فلاپ و رجیستر، حافظه‌ی ROM/RAM، و مفهوم کلاک/سیگنال همگام. اگر در هرکدام مطمئن نیستید، پیش از شروع هفته‌ی اول مرور کنید.

۱-۴ نصب و راه‌اندازی ابزار

۱. Java Runtime Environment (نسخه‌ی ۸ یا بالاتر) را نصب کنید.
۲. آخرین نسخه‌ی Digital را از مخزن رسمی آن دریافت کنید: <https://github.com/hneemann/> digital (بخش Releases).
۳. فایل Digital.jar را اجرا کنید. برای اجرای دستورهای خط‌فرمانی که در این سند می‌بینید، باید بدانید Digital.jar در کجای سیستم شما قرار دارد و از همان مسیر در پنجره‌ی فرمان (Command Prompt یا PowerShell) اجرا شود.
۴. پیشنهاد: پیش از شروع، پوشه‌ی examples همراه نرم‌افزار (از طریق منوی باز کردن نمونه‌ها در برنامه) را باز کنید و پردازنده‌ی نمونه‌ی تک‌سیکل شبیه MIPS که در آن پیدا می‌کنید را مطالعه کنید. این مثال برای الهام گرفتن از سبک طراحی بسیار مفید است (چطور رجیستر فایل ساخته شده، چطور کنترلر کار می‌کند)، اما مجموعه‌دستور آن با PARS-16 یکی نیست و طبق قانون زیر کپی مستقیم آن ممنوع است.

۵-۱ صداقت علمی و قوانین تحویل

ممنوعیت صریح

- import کردن یا کپی کردن مستقیم مدار پردازنده‌ی نمونه‌ی داخلی Digital (یا هر پردازنده‌ی آماده‌ی دیگر پیدا شده در اینترنت) ممنوع است. مجموعه‌دستور PARS-16 به‌اندازه‌ی کافی متفاوت است که چنین کاری عملاً کار نمی‌کند، اما تلاش برای آن نیز خودش تخلف محسوب می‌شود.
- اشتراک‌گذاری فایل dig. ممنوع است.
- دستیاران آموزشی از دستور CLI stats (بخش؟؟) برای انگشت‌نگاری ساختاری مدارهای تحویلی استفاده می‌کنند؛ موارد مشابه برای بررسی دستی پرچم‌گذاری می‌شوند.

۱-۵-۱ قالب تحویل

هر فرد دقیقاً یک فایل NN_CPU.dig (به‌جای NN شماره‌ی دانشجویی) به‌همراه هر زیرمدار کمکی که با import در آن استفاده کرده (همه در یک پوشه‌ی فشرده) تحویل می‌دهد. اگر طراحی سلسله‌مراتبی (چند فایل dig) دارید، تمام فایل‌ها را در یک پوشه‌ی فشرده‌ی واحد با نام NN.zip تحویل دهید تا وابستگی‌های import سالم بمانند (به‌یاد داشته باشید Digital فایل‌های import شده را صرفاً بر اساس نام‌شان نه مسیرشان پیدا می‌کند؛ پس همه‌چیز را کنار هم نگه دارید).

الزام حیاتی: پین‌های ورودی/خروجی فایل بالادستی (یعنی همان NN_CPU.dig) باید دقیقاً با «قرارداد پین» فصل ۳-۳ مطابقت داشته باشند در غیر این صورت هیچ‌کدام از تست‌های خودکار روی مدار شما اجرا نمی‌شوند.

فصل ۲

مرجع کامل مجموعه دستور PARS-16

۱-۲ مشخصات کلی (الزامی و ثابت برای همه‌ی گروه‌ها)

مشخصه	مقدار
عرض کلمه (داده و دستور)	16 bit
تعداد رجیستر عمومی	8 عدد (R7—R0)
معماری حافظه	Harvard — حافظه‌ی دستور و داده کاملاً مجزا
آدرس‌دهی	Word-addressed (هر آدرس = یک کلمه‌ی 16 بیتی، نه بایت)
اندازه‌ی حافظه‌ی دستور (ROM)	دقیقاً 256 کلمه — آدرس 8 bit
اندازه‌ی حافظه‌ی داده (RAM)	دقیقاً 256 کلمه — آدرس 8 bit
تعداد کد عملیات	16 دستور (تمام فضای opcode پر می‌شود)
مسیر داده	تک‌سیکل ^۱ (اجباری برای نمره‌ی کامل)

چرا اندازه‌ی حافظه ثابت است

برخلاف بسیاری از تمرین‌های معماری که اندازه‌ی حافظه را «به انتخاب شما» می‌گذارند، اینجا اندازه‌ی حافظه (و در نتیجه عرض PC) **عمداً ثابت و اجباری** است: تست خودکار بر پایه‌ی پین PC_OUT با عرض دقیق 8 bit کار می‌کند. اگر عرض حافظه‌ی خودتان را تغییر دهید، تست‌های خودکار روی مدار شما اجرا نخواهند شد.

۲-۲ رجیسترها

هشت رجیستر عمومی 16 بیتی با قرارداد زیر:

رجیستر	نام مستعار	کاربرد قراردادی
R0	ZERO	همیشه مقدار 0؛ نوشتن روی آن بی‌اثر است (مثل RISC-V/MIPS)
R1—R6	—	رجیسترهای عمومی، آزاد
R7	RA	رجیستر بازگشت ^۱ ؛ توسط JAL به‌طور خودکار نوشته می‌شود

رجیستر فایل شما باید علاوه بر دو پورت خواندن معمولی داخل مسیر داده (برای Rs1 و Rs2)، یک پورت خواندن سوم فقط برای دیباگ هم داشته باشد (بخش ۳-۳) که هیچ اثری روی اجرای برنامه ندارد و صرفاً برای تست خودکار استفاده می‌شود.

۳-۲ مدل حافظه

معماری Harvard: حافظه‌ی دستور (ROM) و حافظه‌ی داده (RAM) کاملاً جدا هستند و آدرس یکسان در این دو به معنای دو مکان متفاوت است. این ساده‌سازی از دردسرهای معماری von Neumann (تداخل واکشی دستور و دسترسی داده در یک حافظه‌ی مشترک) در مسیر داده‌ی تک‌سیکل جلوگیری می‌کند — دقیقاً همان انتخابی که نسخه‌ی آموزشی تک‌سیکل MIPS در کتاب پترسون-هنسی هم می‌کند [۱].

هیچ مفهوم byte addressing در این معماری وجود ندارد: آدرس 5 یعنی ششمین کلمه‌ی ۱۶ بیتی حافظه، نه بایت پنجم.

۴-۲ رجیستر فلگ و اجرای شرطی سراسری

ویژگی متمایزکننده‌ی PARS-16 این است: هر دستور، بدون استثنا، دو بیت شرط اجرا (CC) دارد. پیش از اجرای واقعی هر دستور، کنترلر بررسی می‌کند آیا شرط آن دستور با وضعیت فعلی رجیستر فلگ برقرار است یا نه. اگر برقرار نباشد، دستور عملاً به یک NOP تبدیل می‌شود: هیچ رجیستری نوشته نمی‌شود، هیچ حافظه‌ای تغییر نمی‌کند، فلگ‌ها دست‌نخورده می‌مانند و حتی پرش/جهش هم رخ نمی‌دهد (PC فقط یک واحد افزایش پیدا می‌کند). این الگو که به آن اجرای پیش‌بینی شده^۱ می‌گویند دقیقاً همان مکانیزمی است که معماری ARM (تا نسخه‌ی ARMv7) روی تمام دستورات خود پیاده می‌کند [۲].

۱-۴-۲ رجیستر فلگ

چهار فلگ زیر در یک رجیستر 4 بیتی نگهداری می‌شوند:

فلگ	نام	شرح
Z	Zero	یک می‌شود اگر نتیجه‌ی آخرین عملیاتِ فلگ‌گذار دقیقاً صفر باشد
N	Negative	برابر با بیت علامت (بیت ۱۵ام) نتیجه است
C	Carry	نقل رقم خروجی جمع‌کننده (برای SUB/ADD)
V	oVerflow	سرریز علامت‌دار (فرمول زیر)

فرمول سرریز برای ADD ($Rd = Rs1 + Rs2$):

$$V = (Rs1_{15} = Rs2_{15}) \text{ و } (Rd_{15} \neq Rs1_{15})$$

¹ Predicated Execution

و برای SUB $(Rd = Rs1 - Rs2)$:

$$V = (Rs1_{15} \neq Rs2_{15}) \text{ و } (Rd_{15} \neq Rs1_{15})$$

برای دستورهای منطقی/شیفت (SHR،SHL،XOR،OR،AND)، همیشه $C=0$ و $V=0$ تعریف می‌شود (فقط Z و N بر اساس نتیجه‌ی نهایی به‌روز می‌شوند).

۲-۴-۲ بیت S: فعال‌سازی به‌روزرسانی فلگ

فقط هفت دستور فرمت R (بخش ۲-۵) یک بیت S در قالب خود دارند. اگر $S=1$ و شرط CC آن دستور برقرار باشد، رجیستر فلگ با نتایج تازه به‌روزرسانی می‌شود؛ اگر $S=0$ باشد (یا شرط برقرار نباشد)، فلگ‌ها دست‌نخورده می‌مانند. هیچ دستور دیگری (SYS،JR،JAL،B،LUI،MOVI،SW،LW،ADDI) اصلاً بیت S در قالب خود ندارد و به‌هیچ‌وجه فلگ عوض نمی‌کند.

۲-۴-۳ کد شرط دوبیتی (CC)

CC	نام	شرط اجرا	کاربرد معمول
00	AL (<i>Always</i>)	همیشه اجرا شود	پیش‌فرض بیشتر دستورها
01	EQ (<i>Equal</i>)	$Z=1$	برابری، پایان حلقه‌ی شمارشی
10	LT (<i>Less Than</i>)	$N=1$	مقایسه‌ی علامت‌دار (کوچک‌تر بودن)
11	VS (<i>oVerflow Set</i>)	$V=1$	تشخیص سرریز حسابی

چرا فقط این سه شرط

با ۲ بیت فقط ۴ حالت داریم که یکی از آن‌ها باید «همیشه» باشد (چون اکثر دستورهای یک برنامه‌ی معمولی اصلاً نمی‌خواهند شرطی باشند). این یعنی فقط ۳ «شرط واقعی» در دسترس است. ما EQ، LT و VS را انتخاب کردیم چون پرکاربردترین دسته‌های مقایسه (برابری، ترتیب،

سرریز) را پوشش می‌دهند. شرط‌های معکوس مثل NE (نامساوی) یا GE (بزرگ‌تر مساوی) به‌طور مستقیم وجود ندارند – در بخش ۲-۴-۶ یاد می‌گیرید چطور آن‌ها را با ترکیب دو دستور شبیه‌سازی کنید. این محدودیتِ عمدی، دقیقاً همان مصالحه‌ای است که طراحان ISA واقعی هم با آن روبه‌رو می‌شوند: عرض کد شرط در برابر تعداد بیت‌های در دسترس برای بقیه‌ی دستور.

۴-۴-۲ مقایسه چگونه انجام می‌شود

هیچ دستور CMP مستقلی در جدول opcode وجود ندارد؛ به‌جایش از SUB با $Rd=R0$ و $S=1$ استفاده می‌شود (نتیجه‌ی تفریق دور ریخته می‌شود چون در $R0$ نوشتن بی‌اثر است، اما فلگ‌ها به‌روز می‌شوند):

```
CMP R1, R2 ; pseudo-op, equivalent to: SUB R0, R1, R2, S
```

۵-۴-۲ الگوی پرش شرطی: چرا دو سیکل کلاک

چون هیچ دستور پرشی اجازه‌ی فعال‌سازی فلگ (بیت S) ندارد، برای «پرش بر اساس یک مقایسه» همیشه به دو دستور پشت‌سرهم نیاز دارید: یکی برای محاسبه و ثبت فلگ، دیگری برای پرشِ شرطی که همان فلگ‌های تازه را می‌خواند:

```
CMP R1, R2 ; cycle 1: update flags based on R1-R2
B.EQ Loop ; cycle 2: if R1==R2, jump to Loop
```

۶-۴-۲ شبیه‌سازی شرط‌های معکوس (NE، GE، ...)

از آنجا که NE به‌طور مستقیم در دسترس نیست، از الگوی «رد شو / پرش بدون‌قید» استفاده کنید. برای مثال، برای پیاده‌سازی «اگر $R1$ با $R2$ برابر نبود به Target برو»:

```

CMP    R1, R2
B.EQ   Skip      ; if equal, skip over the unconditional jump
B.AL   Target     ; otherwise (i.e. not equal), jump unconditionally
Skip:

```

این یک الگوی شناخته‌شده در معماری‌های اولیه‌ای است که مجموعه‌ی محدودی از کدهای شرط داشتند؛ تمرین آن به شما کمک می‌کند محدودیت‌های سخت‌افزاری واقعی را از نزدیک لمس کنید.

۵-۲ فرمت‌های دستور

فیلد CC (دو بیت، بیت‌های 14-15) و فیلد OPCODE (چهار بیت، بیت‌های 10-13) در تمام فرمت‌ها دقیقاً در همین موقعیت ثابت قرار دارند این یکنواختی، رمزگشایی سیگنال‌های کنترلی را به‌شدت ساده می‌کند.

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	Rd	Rs1	Rs2	S

فرمت R – عملیات ثابت‌به‌ثبات (۷ دستور، تنها فرمتی با بیت S)

MSB (bit 15)		LSB (bit 0)		
CC	OPCODE	Rd/Rs2	Rs1	imm4

فرمت I – عملیات با مقدار فوری کوچک (بدون فلگ)

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	Rd	imm7

فرمت U – بارگذاری مقدار فوری بزرگ‌تر

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	offset10	

فرمت B/J – پرش نسبی (خودِ CC شرط پرش را می‌سازد؛ نیازی به فیلد رجیستر نیست)

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	Rs1	reserved

فرمت JR – پرش غیرمستقیم به آدرس داخل یک رجیستر

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	func	reserved

فرمت SYS – دستورهای ویژه (HALT/NOP)

نکته‌ی مهم درباره‌ی فرمت I: در دستور SW، فیلدی که در LW/ADDI به‌عنوان «مقصد نوشتن» (Rd) خوانده می‌شود، در SW به‌عنوان «رجیستر منبع داده برای نوشتن در حافظه» (Rs2) تفسیر می‌شود – موقعیت بیتی یکسان است، فقط معنایش فرق می‌کند. این دقیقاً همان ترفندی است که RISC-V هم در فرمت S خود استفاده می‌کند.

۶-۲ جدول کامل دستورات

عملکرد	S	فرمت	Mnemonic	Opcode
$Rd = Rs1 + Rs2$	بله	R	ADD	0000
$Rd = Rs1 - Rs2$ (پایه‌ی CMP)	بله	R	SUB	0001
$Rd = Rs1 \& Rs2$	بله	R	AND	0010
$Rd = Rs1 Rs2$	بله	R	OR	0011
$Rd = Rs1 \wedge Rs2$	بله	R	XOR	0100
شیفت منطقی چپ $Rs1$: مقدار شیفت $Rs2[3:0] =$	بله	R	SHL	0101
شیفت منطقی راست $Rs1$: مقدار $Rs2[3:0] =$ شیفت	بله	R	SHR	0110
$Rd = Rs1 + sext(imm4)$	خیر	I	ADDI	0111
$Rd = MEM[Rs1 + sext(imm4)]$	خیر	I	LW	1000
$MEM[Rs1 + sext(imm4)] = Rs2$	خیر	I	SW	1001
$Rd = sext(imm7)$ (بارگذاری مستقیم ثابت کوچک)	خیر	U	MOVI	1010
$Rd[8:0] = 0, Rd[15:9] = imm7$	خیر	U	LUI	1011
اگر CC برقرار: $PC = PC + 1 + sext(offset10)$	خیر	B	B	1100
اگر CC برقرار: $PC = PC + 1 + sext(offset10)$	خیر	J	JAL	1101
اگر CC برقرار: $PC = Rs1$ (بازگشت از زیرروال با $R7$ JR)	خیر	JR	JR	1110
$func=0001$: NOP $func=0000$: HALT (HALT_OUT=1)	خیر	SYS	SYS	1111

تمام آفست‌ها (imm4، imm7 در MOV، offset10) به صورت علامت‌دار دوماً (two's complement) تفسیر می‌شوند مگر آنکه خلاف آن گفته شود. آفست پرش نسبت به PC+1 (یعنی آدرس دستور بعدی) محاسبه می‌شود.

۷-۲ شبه‌دستورها

یک اسمبلر کمکی می‌تواند شبه‌دستورهای زیر را به دستورهای واقعی جدول بالا ترجمه کند:

شبه‌دستور	معادل واقعی	توضیح
CMP Rs1, Rs2	SUB R0, Rs1, Rs2, S	مقایسه بدون نوشتن نتیجه
MOV Rd, Rs	ADD Rd, Rs, R0	کپی مقدار بین رجیسترها
NOP	func=0000 با SYS	بدون عملیات
HALT	func=0001 با SYS	توقف اجرا (HALT_OUT=1)
RET	JR R7	بازگشت از زیرروال
LI Rd, #imm16	ADDI+LUI (تا ۳ دستور)	بارگذاری ثابت ۱۶ بیتی دلخواه

۸-۲ نحو زبان اسمبلی

- هر خط: یک برچسب اختیاری (Label:)، یک دستور یا شبه‌دستور، و یک توضیح اختیاری بعد از ;.
- کد شرط به صورت پسوند با نقطه می‌آید: B.EQ، B.LT، B.VS؛ نبود پسوند یعنی AL (بدون قید).
- فعال‌سازی فلگ با پسوند S، در انتهای دستورهای فرمت R: مثلاً ADD R1, R2, R3, S.
- اعداد: دهدهی معمولی (12)، مبنای شانزده با پیشوند 0x (0x1F)، دودویی با پیشوند 0b.

- نشانی دهی حافظه در SW/LW به سبک $\text{offset}(\text{base})$: مثلاً $4(R2)$ ، یعنی $R1 = \text{MEM}[R2+4]$ ، و $\text{MEM}[R2-2] = R3$ یعنی SW $R3, -2(R2)$.

۱-۸-۲ مثال کامل: جمع اعداد 1 تا N

```

MOVI R1, 10      ; R1 = N = 10
MOVI R2, 0       ; R2 = sum = 0
MOVI R3, 1       ; R3 = i = 1

Loop:  CMP    R3, R1
      B.LT Body      ; if i < N, continue (see note below)
      B.EQ Body      ; the i == N case must also run once more
      B.AL Done

Body:  ADD    R2, R2, R3 ; sum += i
      ADDI   R3, R3, 1  ; i++
      B.AL Loop

Done:  SW     R2, 0(R0) ; store the result in MEM[0]

      HALT

```

این مثال به خوبی نشان می‌دهد که نبود شرط LE (کوچک‌تر یا مساوی) مستقیم، برنامه‌نویسی را کمی پرحرف‌تر می‌کند دقیقاً همان بده‌بستانی که در بخش ۲-۴-۶ توضیح داده شد.

فصل ۳

راهنمای معماری میکرو و قرارداد رابط اجباری

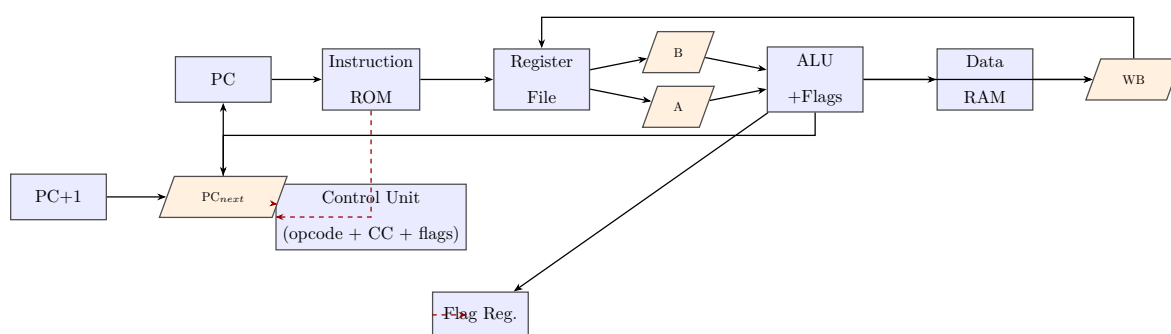
۱-۳ مسیر داده‌ی تک‌سیکل

برای بازه‌ی سه‌هفته‌ای، مسیر داده‌ی تک‌سیکل^۱ الزام پایه است: هر دستور دقیقاً در یک پالس کلاک، از واکنشی تا نوشتن نتیجه، کامل اجرا می‌شود.

۱-۱-۳ نمای کلی بلوکی

شکل زیر ارتباط اجزای اصلی را نشان می‌دهد (جهت سیگنال‌های کنترلی با خط‌چین قرمز مشخص شده است). این یک نقشه‌ی مفهومی است، نه سیم‌کشی دقیق جزئیات هر باس و هر مالتی‌پلکسر را خودتان طراحی می‌کنید.

¹ Single-Cycle



۲-۱-۳ ترتیب پیشنهادی ساخت اجزا (مطابق با فصل ۲-۲)

۱. PC (رجیستر 8 bit) + جمع‌کننده‌ی +1
۲. حافظه‌ی دستور – یک قطعه‌ی ROM با آدرس 8 bit و خروجی 16 bit
۳. رجیستر فایل 8×16 (دو پورت خواندنِ عادی + یک پورت نوشتن + یک پورت خواندنِ سوم برای دیباگ، بخش ۳-۳)
۴. واحد بسط علامت^۱ برای imm4، imm7 و offset10 (سه ورودی متفاوت با عرض‌های متفاوت، خروجی همیشه 16 bit)
۵. ALU با پشتیبانی از هفت عملیات جدول ۲-۶ و محاسبه‌ی هم‌زمان چهار فلگ
۶. رجیستر فلگ (فقط زمانی بارگذاری می‌شود که FlagWrite=1 – بخش ۲-۳)
۷. واحد ارزیابی شرط: یک مدار ترکیبیِ کوچک که CC فعلی و رجیستر فلگ را می‌خواند و یک بیت cond_met تولید می‌کند
۸. حافظه‌ی داده RAM با آدرس 8 bit
۹. واحد کنترل (بخش ۲-۳)
۱۰. مالتی پلکس‌های عملوند/نتیجه/آدرس بعدی + گیت‌کردن سیگنال‌های نوشتن با cond_met
۱۱. پورت‌های دیباگ (بخش ۳-۳) – در پایان، اما پیش از تحویل هفته‌ی دوم

¹ Sign-Extend

۲-۳ جدول سیگنال‌های کنترلی

این جدول طرح مرجع پیشنهادی است (می‌توانید نام سیگنال‌ها را عوض کنید، اما رفتار باید معادل باشد). ردیف‌ها بر اساس گروه دستور خلاصه شده‌اند چون دستورهای هم‌فرمت، کنترل یکسانی دارند (فقط عملکرد ALU در فرمت R فرق می‌کند که مستقیماً برابر با سه بیت پایینِ OPCODE است).

گروه	RegW	RegDst	MemRd	MemWr	ALUSrcA	ALUSrcB	ALUFunc	WBSel	PCSrc
SHR..ADD (0000-0110)	1	Rd	0	0	Rs1	Rs2	=OPCODE[2:0]	ALU	seq
ADDI (0111)	1	Rd	0	0	Rs1	imm4	ADD	ALU	seq
LW (1000)	1	Rd	1	0	Rs1	imm4	ADD	Mem	seq
SW (1001)	0	–	0	1	Rs1	imm4	ADD	–	seq
MOVI (1010)	1	Rd	0	0	Zero	imm7	ADD	ALU	seq
LUI (1011)	1	Rd	0	0	–	–	–	LUI	seq
B (1100)	0	–	0	0	–	–	–	–	Branch
JAL (1101)	1	R7	0	0	–	–	–	PC+1	JAL
JR (1110)	0	–	0	0	–	–	–	–	JR
SYS (1111)	0	–	0	0	–	–	–	–	seq

راهنمای ستون‌ها: RegW: نوشتن روی رجیستر فایل مجاز است؟ RegDst: آدرس مقصد نوشتن از کجا می‌آید (فیلد Rd، یا همیشه R7 برای JAL)؟ ALUSrcB/ALUSrcA: ورودی‌های ALU از کجا می‌آیند؟ WBSel: مقداری که در رجیستر فایل نوشته می‌شود از کدام منبع است (خروجی ALU، داده‌ی حافظه، نتیجه‌ی الحاق LUI، یا PC+1 برای JAL)؟ PCSrc: مقدار بعدی PC (توالی عادی PC+1، هدف پرش نسبی B، هدف JAL، یا محتوای Rs1 برای JR)؟

نکته‌ی حیاتی: گیت کردن با شرط

هر مقدار «۱» در ستون‌های RegW، MemWr و هر انتخاب غیر از «توالی عادی» در PCSrc، باید علاوه‌بر جدول بالا، با بیت `cond_met` هم AND شود. یعنی:

$$\text{RegW_final} = \text{RegW_table} \text{ AND } \text{cond_met}$$

و مشابه برای `MemWr_final` و انتخاب `JR/JAL/Branch` در PCSrc (اگر شرط برقرار نبود، PCSrc عملاً به «توالی عادی» برمی‌گردد،

حتی اگر دستور از نوع JR/JAL/B بوده باشد). به‌همین ترتیب: $\text{FlagWrite_final} = \text{S_bit}$ AND FlagCapable AND cond_met (که FlagCapable یعنی این دستور از گروه فرمت R است ستون آخر جدول قبل).

۳-۲-۱ پیشنهاد برای ساخت خود واحد کنترل

ساده‌ترین و تمیزترین روش: یک قطعه‌ی ROM کوچک که با OPCODE (۴ بیت) آدرس‌دهی می‌شود و محتویات هر سطر آن، دقیقاً همان سیگنال‌های کنترلی جدول بالا برای آن opcode است (الگوی کلاسیک «control store»). این را می‌توانید با ادیتور داده‌ی ROM در Digital به‌سادگی پر کنید. سپس یک لایه‌ی ترکیبی کوچک، خروجی این ROM را با AND cond_met می‌کند تا سیگنال‌های نهایی به‌دست بیایند.

۳-۳ قرارداد پین الزام سخت برای نمره‌دهی خودکار

این بخش را با دقت بخوانید

اگر مدار نهایی شما (NN_CPU.dig) دقیقاً همین پین‌ها را با همین نام و همین عرض نداشته باشد، هیچ‌کدام از تست‌های خودکار روی آن اجرا نخواهند شد و نمره‌ی آن بخش صفر ثبت می‌شود (نه به‌خاطر اشتباه در طراحی، بلکه چون ابزار تست اصلاً نمی‌تواند پین‌ها را پیدا کند). نام‌ها به بزرگی/کوچکی حروف حساس‌اند.

پین	جهت	عرض	توضیح
CLK	ورودی	1	کلاک سیستم
RESET	ورودی	1	ریست همگام، فعال در 1 (باید PC، رجیسترها و فلگ‌ها را صفر کند)
DBG_RSEL	ورودی	3	شماره‌ی رجیستری که می‌خواهیم از پورت دیباگ ببینیم
PC_OUT	خروجی	8	مقدار جاری PC
IR_OUT	خروجی	16	دستور در حال واکنشی/اجرا (خروجی خام حافظه‌ی دستور)
FLAGS_OUT	خروجی	4	ترتیب بیت‌ها: {V, C, N, Z} (V=بیت ۳ ... Z=بیت ۰)
DBG_ROUT	خروجی	16	مقدار رجیستر انتخاب‌شده با DBG_RSEL (پورت خواندن سوم و مستقل رجیستر فایل)
HALT_OUT	خروجی	1	پس از اجرای موفق HALT، یک می‌شود و تا RESET بعدی یک باقی می‌ماند

این «پورت دیباگ» هیچ اثری روی درستی معماری ندارد صرفاً یک پنجره‌ی مشاهده برای تست خودکار است، دقیقاً مثل پورت دیباگ/JTAG در پردازنده‌های واقعی. پیاده‌سازی داخلی شما (نام سیم‌ها، تعداد فایل، ساختار سلسله‌مراتبی) کاملاً آزاد است؛ فقط این هشت پین در بالادستی‌ترین فایل باید دقیقاً همین‌طور باشند.

۳-۳-۱ نکته‌ی پیاده‌سازی: پورت خواندن سوم رجیستر فایل

اکثر رجیستر فایل‌هایی که در آموزش می‌سازید، دو پورت خواندن دارند (برای Rs1 و Rs2). برای پورت دیباگ، یک پورت خواندن سوم و کاملاً مستقل لازم دارید که آدرسش از DBG_RSEL (نه از

فیلدهای دستور) می‌آید. ساده‌ترین راه: از همان حافظه/آرایه‌ی رجیستری که قبلاً ساخته‌اید، یک خروجی multiplexer دیگر هم بگیرید که با DBG_RSEL انتخاب می‌شود کاملاً موازی و بدون تداخل با دو پورت خواندن اصلی.

۳-۴ سؤالات متداول طراحی

آیا استفاده از قطعه‌ی آماده‌ی Arithmetic Multiplexer یا کتابخانه‌ی Digital مجاز است؟ بله. تمرکز این پروژه بر معماری و سازمان‌دهی سطح بالا است، نه طراحی گیتی ALU. استفاده از قطعات آماده‌ی کتابخانه (که خودشان مدار دیجیتال هستند، نه کد نوشته‌شده) کاملاً مجاز است.

آیا می‌توانم مدار را به صورت چندفایلی (سلسله‌مراتبی) بسازم؟ بله، و توصیه هم می‌شود (رجیستر فایل، ALU و کنترلر را در فایل‌های جدا بسازید). فقط یادتان باشد طبق بخش ۱-۵، همه‌ی فایل‌ها باید در تحویل نهایی همراه هم باشند.

اگر بخواهم عرض حافظه را عوض کنم چه می‌شود؟ نکنید. عرض 8 bit برای PC و آدرس حافظه بخشی از قرارداد اجباری است (بخش ۳-۳).

Bibliography

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [2] ARM Limited, *ARM Architecture Reference Manual, ARMv7-A and ARMv7-R Edition*, 2014, conditional (predicated) instruction execution, Chapter A8.
- [3] H. Neemann, “Digital: A digital logic designer and circuit simulator,” <https://github.com/hneemann/Digital>, 2026, accessed July 2026.
- [4] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, 2024.